Name of Student: Minisha Singh	**PRN: 25070122292**
Semester: III	**Year: AY 2026-27**
Subject: Programming with Java	
ASSIGNMENT No: 28	
TITLE: Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication	

Problem Statement

Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Learning Objectives

To implement Remote Method Invocation for distributed Java applications.

Programme

1. Create a remote calculator service using Java RMI

Remote interface code:

```
import java.rmi.*;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int subtract(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
    double divide(int a, int b) throws RemoteException;
}
```

Calculator Implementation Code:

```
import java.rmi.*;
import java.rmi.server.*;

public class CalculatorImpl extends UnicastRemoteObject implements
Calculator {
    public CalculatorImpl() throws RemoteException {
        super();
    }

    public int add(int a, int b) {
```

```
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public double divide(int a, int b) throws RemoteException {
        if (b == 0) {
            throw new RemoteException("Cannot divide by zero");
        }
        return (double) a / b;
    }
}
```

Server Code:

```
import java.rmi.registry.*;

public class CalculatorServer {
    public static void main(String[] args) {
        try {
            Calculator service = new CalculatorImpl();

            Registry registry = LocateRegistry.createRegistry(1099);
            registry.rebind("CalculatorService", service);

            System.out.println("Calculator server is running.");
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Client Code:

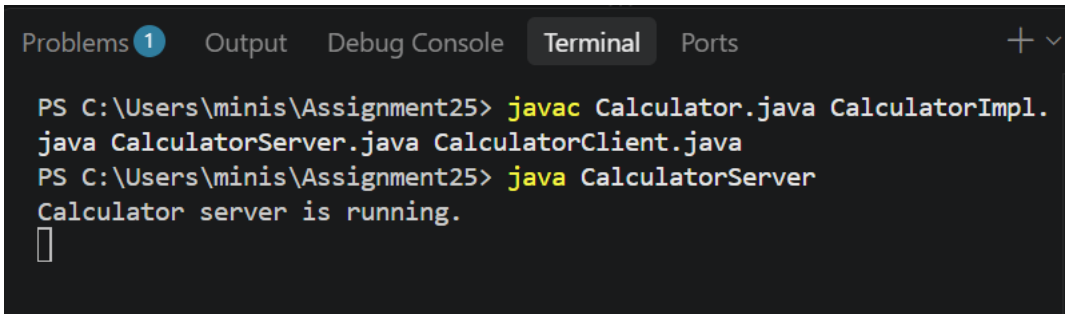
```
import java.rmi.registry.*;

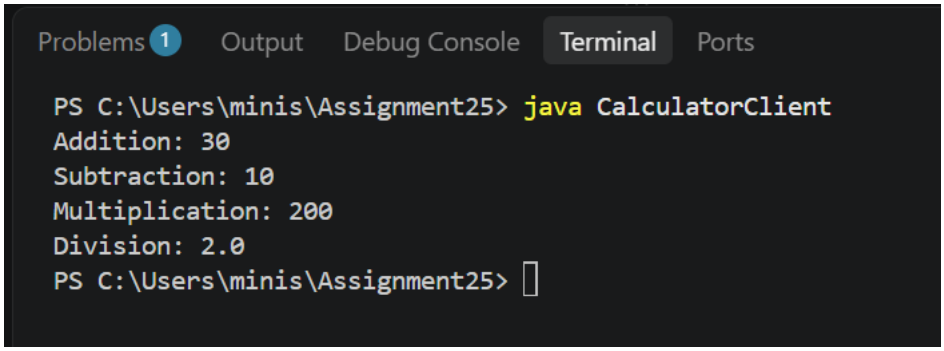
public class CalculatorClient {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
```

```
            Calculator service = (Calculator) registry.lookup("CalculatorService");

            System.out.println("Addition: " + service.add(20, 10));
            System.out.println("Subtraction: " + service.subtract(20, 10));
            System.out.println("Multiplication: " + service.multiply(20, 10));
            System.out.println("Division: " + service.divide(20, 10));
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output:





2. Create a Java application using RMI where the server converts temperature values between Celsius and Fahrenheit. The client sends input temperature and receives converted output.

Temperature interface code:

```
import java.rmi.*;

public interface TemperatureConverter extends Remote {
    double celsiusToFahrenheit(double celsius) throws RemoteException;
    double fahrenheitToCelsius(double fahrenheit) throws RemoteException;
}
```

Temperature Implementation Code:

```
import java.rmi.*;
import java.rmi.server.*;

public class TemperatureConverterImpl extends UnicastRemoteObject
        implements TemperatureConverter {

    public TemperatureConverterImpl() throws RemoteException {
        super();
    }

    public double celsiusToFahrenheit(double celsius) {
        return (celsius * 9 / 5) + 32;
    }

    public double fahrenheitToCelsius(double fahrenheit) {
        return (fahrenheit - 32) * 5 / 9;
    }
}
```

Server Code:

```
import java.rmi.registry.*;

public class TemperatureServer {
    public static void main(String[] args) {
        try {
            TemperatureConverter service = new TemperatureConverterImpl();

            Registry registry = LocateRegistry.createRegistry(1098);
            registry.rebind("TemperatureService", service);

            System.out.println("Temperature server is running.");
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Client Code:

```
import java.rmi.registry.*;
import java.util.Scanner;

public class TemperatureClient {
```

```
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1098);

            TemperatureConverter service =
                (TemperatureConverter) registry.lookup("TemperatureService");

            Scanner sc = new Scanner(System.in);

            System.out.print("Enter temperature in Celsius: ");
            double celsius = sc.nextDouble();

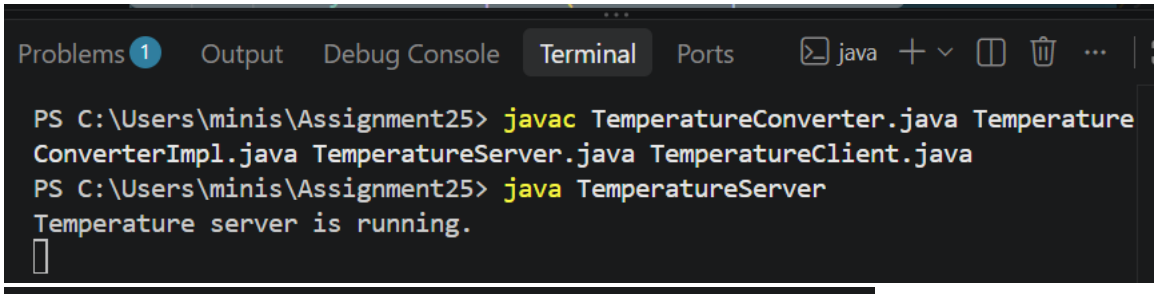
            System.out.println("Fahrenheit: "
                + service.celsiusToFahrenheit(celsius));

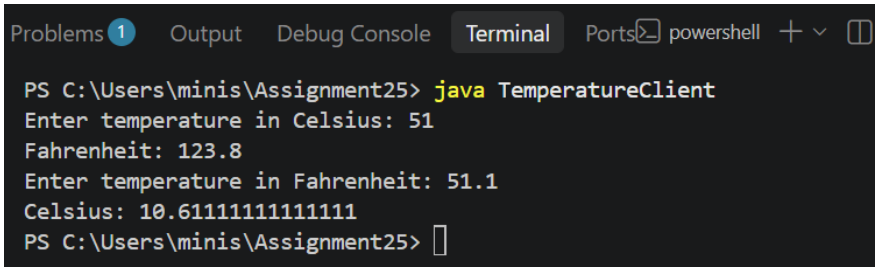
            System.out.print("Enter temperature in Fahrenheit: ");
            double fahrenheit = sc.nextDouble();

            System.out.println("Celsius: "
                + service.fahrenheitToCelsius(fahrenheit));

            sc.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output:





Conlclusion:

This assignment helped us understand Remote Method Invocation (RMI) in Java and how remote methods can be called between a client and a server. We implemented a remote calculator to perform arithmetic operations and a temperature converter to convert Celsius to Fahrenheit and vice versa. We learned about remote interfaces, implementation classes, RMI registries, and client-server communication. Overall, this assignment improved our understanding of distributed Java applications and remote method execution.

Conclusion